Contents

1 Basic 1
- 1.1 default . . . 1
- 1.2 奇妙的東西 . . . 1
- 1.3 random . . . 1
- 1.4 run.bat . . . 1
- 1.5 run.sh . . . 1
- 1.6 對拍屁眼扣 . . . 1

2 Binary Search 2
- 2.1 二分搜 . . . 2
- 2.2 三分搜 . . . 2

3 DataStructure 2
- 3.1 DSU(並查集) . . . 2
- 3.2 FenwickTree/BIT . . . 2
- 3.3 二維 BIT . . . 3
- 3.4 Seg/線段樹 . . . 3
- 3.5 李超線段樹 . . . 3
- 3.6 SegTree Beats/線段樹 Beats . . . 4

4 Graph 4
- 4.1 Topo/拓樸 . . . 4
- 4.2 bellman/邊長鬆弛 . . . 5
- 4.3 ola/歐拉 . . . 5
- 4.4 LCA/最近公共祖先 . . . 5
- 4.5 Scc/強連通分量 . . . 5
- 4.6 Bcc/雙連通分量 . . . 5
- 4.7 最大團 . . . 6
- 4.8 平均最小團 . . . 6
- 4.9 支配點 . . . 7

5 dynamic programming 7
- 5.1 SOS DP . . . 7
- 5.2 p-median . . . 7

6 Computational Geometry 7
- 6.1 defult . . . 7
- 6.2 Circle . . . 7
- 6.3 極座標排序 . . . 8
- 6.4 ConvexHull/凸包 (傑) . . . 8
- 6.5 ConvexHull/凸包 (宏) . . . 8
- 6.6 PtSegDist . . . 8
- 6.7 兩線相交 . . . 8
- 6.8 點是否在多邊形內 . . . 8
- 6.9 最遠點對/旋轉卡尺 . . . 8
- 6.10 最近點對 . . . 9
- 6.11 最小覆蓋圓 . . . 9

7 Flow 9
- 7.1 km/二分圖最大權匹配 . . . 9
- 7.2 Dinic . . . 10
- 7.3 hungarian/匈牙利演算法 . . . 10
- 7.4 SPFA 最小費流 . . . 10
- 7.5 差分約束系統 . . . 10

8 String 11
- 8.1 KMP . . . 11
- 8.2 Z-value . . . 11
- 8.3 馬拉車 . . . 11
- 8.4 minRotation/最小字典序旋轉 . . . 11
- 8.5 SAIS . . . 11
- 8.6 Trie/字典樹 . . . 12
- 8.7 ACautomata/AC 自動機 . . . 12

9 Math 12
- 9.1 basic . . . 12
- 9.2 快速冪 . . . 12
- 9.3 大數運算 . . . 13
- 9.4 Prime . . . 13
- 9.5 ExGCD . . . 13
- 9.6 Phi . . . 14
- 9.7 Euler/歐拉函數 . . . 14
- 9.8 miller rabin/判斷質數 . . . 14
- 9.9 Pollard Rho/質因數分解 . . . 14
- 9.10 FFT/傅立葉轉換 (多項式相乘) . . . 14
- 9.11 josephus/約瑟夫問題 . . . 15
- 9.12 指數塔 . . . 15
- 9.13 BSGS . . . 15
- 9.14 $x^2 \pmod p == a$ . . . 15
- 9.15 Theorem . . . 15

10 酷東西可能會用到 16
- 10.1 areaofrect/矩形聯集面積 . . . 16

1 Basic

1.1 default

```cpp
#include <bits/stdc++.h>
using namespace std;
#define int long long
#define idonthavegirlfriend ios::sync_with_stdio(0),cin
    .tie(0);
#define pb push_back
#define endl "\n"
#define all(v) (v).begin(),(v).end()
void solve(){

}
signed main(){
    idonthavegirlfriend
    int _=1;
    //cin>>_;
    while(_--){
        solve();
    }
}
```

1.2 奇妙的東西

```cpp

priority_queue<T> //由大到小
priority_queue<T,vector<T>,greater<T>> //由小到大
__builtin_popcountll// 二進位有幾個1(記得這是long long)
__builtin_clzll          // 左起第一個1之前0的個數
__builtin_parityll       // 1的個數的奇偶性
__builtin_mul_overflow(a,b,&h) // a*b是否溢位,h = a * b
    ;
__builtin_add_overflow(a,b,&h)
```

1.3 random

```cpp
mt19937 mt(chrono::steady_clock::now().time_since_epoch
    ().count());
//mt19937_64 mt() -> return randnum
int randint(int l, int r){
    uniform_int_distribution<> dis(l, r); return dis(mt
        );
}
```

1.4 run.bat

```bat
@echo off
g++ ac.cpp -o ac.exe
g++ wa.cpp -o wa.exe
set /a num=1
:loop
   echo %num%
   python gen.py > input
   ac.exe < input > ac
   wa.exe < input > wa
   fc ac wa
   set /a num=num+1
if not errorlevel 1 goto loop
```

1.5 run.sh

```sh
set -e
for ((i=0;;i++))
do
    echo "$i"
    python gen.py > in
    ./ac < in > ac.out
    ./wa < in > wa.out
    diff ac.out wa.out || break
done
```

1.6 對拍屁眼扣

```python
from random import *

n = randint(1, 100)   # 隨機產生 1~100的整數
ch = chr(ord('a') + randint(0, 25)) # 隨機產生 'a'~'z'
    其中一個字元
choiceSet = sample(s, 4)   # 從集合s選出4個不同的元素
```

```
choiceSet = sample(range(1, n+1), 4)  #從整數 1~n 選出
    4 個不同的元素
shuffle(arr)  # 把序列 arr 順序打亂
# 隨機產生一組 1~n 的 permutation
from random import *
n = randint(3, 6)
arr = [i+1 for i in range(n)]
shuffle(arr)
print(n)
print(*arr)
s = ""
n = randint(1, 10)
# 隨機產生一個長度為 n 的小寫字母字串
for i in range(n):
    s += chr(ord('a') + randint(0, 25))
print(s)
#產生樹
from random import *
n = randint(3, 6)
print(n)
for i in range(2, n+1):
    print(randint(1, i-1), i)
#隨機產生無自環無重邊的一張連通圖
from random import *
n = randint(5, 10)
m = randint(n-1, n+3)
print(n, m)
edge = list()
#construct tree
for i in range(2, n+1):
    x = randint(1, i-1)
    y = i
    edge.append([min(x, y), max(x, y)])
    print(x, y)
#add extra edge
for i in range(m-(n-1)):
    x = randint(1, n)
    y = randint(1, n)
    while x == y or [min(x, y), max(x, y)] in edge:
        x = randint(1, n)
        y = randint(1, n)
    print(x, y)
    edge.append([min(x, y), max(x, y)])
```

2 Binary Search

2.1 二分搜

```
int bsearch_1(int l, int r)
{
    while (l < r)
    {
        int mid = l + r >> 1;
        if (check(mid)) r = mid;
        else l = mid + 1;
    }
    return l;
}
// ................oooooooooo
int bsearch_2(int l, int r)
{
    while (l < r)
    {
        int mid = l + r + 1 >> 1;
        if (check(mid)) l = mid;
        else r = mid - 1;
    }
    return l;
}
// oooooooooo.................
int m = *ranges::partition_point(views::iota(0LL,(int)1
    e9+9),[&](int a){
    return check(a) > k;
    });
//[begin,last)
//11111110000000000
//搜左邊數過來第一個 0
//都是 1 會回傳 last
int partitionpoint(int L,int R,function<bool(int)> chk)
    {
  int l = L,r = R-1;
  while(r - l > 10){
    int m = l + (r-l)/2;
    if(chk(m)) l = m;
    else r = m;
  }
    int m = l;
    while(m <= r){
        if(!chk(m)) break;
        ++m;
    }
  if(!chk(m)) return m;
  else return R;
}
//手工
```

2.2 三分搜

```
int l = 1,r = 100;
while(l < r) {
    int lmid = l + (r - l) / 3; // l + 1/3區間大小
    int rmid = r - (r - l) / 3;  // r - 1/3區間大小
    lans = cal(lmid),rans = cal(rmid);
    // 求凹函數極小直
    if(lans <= rans) r = rmid - 1;
    else l = lmid + 1;
}
```

3 DataStructure

3.1 DSU(並查集)

```
struct DSU {
    vector<int> f, sz;
    int cnt;
    DSU(int n) : f(n), sz(n),cnt(n) {
        for (int i = 0; i < n; i++) {
            f[i] = i;
            sz[i] = 1;
        }
    }
    int find(int x) {
        if (x == f[x]) return x;
        return f[x] = find(f[x]);
    }
    void merge(int x, int y) {
        x = find(x); y = find(y);
        if (x == y) return;
        if (sz[x] < sz[y])
            swap(x, y);
        f[y] = x;
        sz[x] += sz[y];
        cnt--;
    }
    bool same(int a, int b) {
        return find(a) == find(b);
    }
    int groups() {//找有幾群
        return cnt;
    }
};
```

3.2 FenwickTree/BIT

```
struct Fenwick {//這個是1base的
    int n;
    vector<int> s;
    int lowbit(int x) { return x & -x; }
    Fenwick(int _n) {
        n = _n + 1;
        s.clear(), s.resize(n, 0);
    }
    void update(int i, int v) {  // 更新數值
        for (; i < n; i += lowbit(i))
            s[i] += v;
    }
    int query(int x) {  // 查詢前綴和(有包含)
        int pre = 0;
        for (; x; x -= lowbit(x))
            pre += s[x];
        return pre;
    }
```

```
    Fenwick(vector<int> a) {// 建構資料結構
        n = a.size(); // a 是 0-based
        s.clear(), s.resize(n + 1, 0); // s 是 1-based
        for (int i = 1; i <= n; i++)
            update(i, a[i - 1]);
    }
};
//使用:
Fenwick fw(a);//a is vector
Fenwick fw(n);//n is int
```

3.3 二維 BIT

```
struct BIT {
  int n;
  vector<vector<ll>> bit;
  int lowbit(int x) { return x & -x; }
  BIT(int _n) : n(_n + 1) {
    bit.assign(n, vector<ll>(n, 0));
  }
  void update(int x, int y, ll val) {  // 更新 (x, y)
    for (int i = x; i < n; i += lowbit(i))
      for (int j = y; j < n; j += lowbit(j))
        bit[i][j] += val;
  }
  ll query(int x, int y) {  // 查詢 (1, 1) ~ (x, y) 的
      和
    ll ans = 0;
    for (int i = x; i; i -= lowbit(i))
      for (int j = y; j; j -= lowbit(j)) ans += bit[i][
          j];
    return ans;
  }
  ll range_query(int a, int b, int x, int y) {
    return query(x, y) - query(x, b - 1) -
           query(a - 1, y) + query(a - 1, b - 1);
  }
};
```

3.4 Seg/線段樹

```
struct Seg {
#define cl(x) (x << 1)
#define cr(x) (x << 1) + 1
    int _n;
    vector<int> f, tag;
    vector<int> arr;
    //Seg(int n) : _n(n), f(4 * n), tag(4 * n) {}
    Seg(vector<int> a) {
        _n = a.size();
        f.resize(4 * _n + 1, 0);
        tag.resize(4 * _n + 1, 0);
        arr.resize(_n);
        for (int i = 0; i < _n; i++) {
            arr[i] = a[i];
        }

        build(1, 0, _n - 1);
    }
    Seg(int x){
        _n = x;
        f.resize(4 * _n + 1, 0);
        tag.resize(4 * _n + 1, 0);
        arr.resize(_n);

    }

    void build(int id, int l, int r) {
        if (l == r) {
            f[id] = arr[l];
            return;
        }
        int mid = (l + r) >> 1;
        build(cl(id), l, mid);
        build(cr(id), mid + 1, r);
        pull(id, l, r);
    }

    void pull(int id, int l, int r) {
        int mid = (l + r) >> 1;
        push(cl(id), l, mid);
        push(cr(id), mid + 1, r);
        f[id] = f[cl(id)] + f[cr(id)];
    }

    void push(int id, int l, int r) {
        if (tag[id]) {
            f[id] += tag[id] * (r - l + 1);
            if (l != r) {
                tag[cl(id)] += tag[id];
                tag[cr(id)] += tag[id];
            }
            tag[id] = 0;
        }
    }

    void update(int id, int l, int r, int x, int v) {
        if (l == r) {
            f[id] = v;
            return;
        }
        int mid = (l + r) >> 1;
        if (x <= mid) update(cl(id), l, mid, x, v);
        if (mid < x)  update(cr(id), mid + 1, r, x, v);
        pull(id, l, r);
    }

    void update(int id, int l, int r, int ql, int qr,
        int v) {
        push(id, l, r);
        if (ql <= l and qr >= r) {
            tag[id] += v;
            return;
        }
        int mid = (l + r) >> 1;
        if (ql <= mid) {
            update(cl(id), l, mid, ql, qr, v);
        }
        if (qr > mid) {
            update(cr(id), mid + 1, r, ql, qr, v);
        }
        pull(id, l, r);
    }

    int query(int id, int l, int r, int sl, int sr) {
        push(id, l, r);
        if (sl <= l and sr >= r) {
            return f[id];
        }
        int res = 0;
        int mid = (l + r) >> 1;
        int ll = 0;
        int rr = 0;
        if (sl <= mid) {
            ll = query(cl(id), l, mid, sl, sr);
        }
        if (sr > mid) {
            rr = query(cr(id), mid + 1, r, sl, sr);
        }
        res = ll + rr;
        return res;
    }
    int find_kth(int id, int l, int r, int k) {
        push(id, l, r);
        if (l == r) return l;
        int mid = (l + r) >> 1;
        push(cl(id), l, mid);
        push(cr(id), mid + 1, r);
        if (f[cl(id)] >= k) {
            return find_kth(cl(id), l, mid, k);
        }
        return find_kth(cr(id), mid + 1, r, k - f[cl(id
            )]);
    }
};
```

3.5 李超線段樹

```
//線段樹套上凸包
//我相信我們賽中想得出來
const int inf = numeric_limits<i64>::max()/2;
struct Line {
    // y = ax + b
    i64 a{0}, b{-inf};
```

```
    i64 operator()(i64 x) {
        return a * x + b;
    }
};

struct Seg{
    int l, r;
    Seg *ls{},*rs{};
    Line f{};
    Seg(int l, int r) : l(l), r(r) {}
    void add(Line g){
        int m = (l+r)/2;
        if (g(m) > f(m)) swap(g, f);
        if(g.b == -inf || r - l == 1) return;
        if(g.a < f.a){
            if(!ls) ls = new Seg(l,m);
            ls->add(g);
        }else{
            if(!rs) rs = new Seg(m,r);
            rs->add(g);
        }
    }
    i64 qry(i64 x){
        int m = (l+r) / 2;
        i64 y = f(x);
        if(x <  m && ls) y = max({y,ls->qry(x)});
        if(x >= m && rs) y = max({y,rs->qry(x)});
        return y;
    }
};
auto add = [&](Line g,int ql,int qr){ //新增線段 [ql,qr
    )
    auto find = [&](auto &&self,Seg * now,int l,int r)
        -> void {
        if(ql <= l && r <= qr){
            now->add(g);
            return;
        }
        int m = (l+r) / 2;
        if(ql < m) {
            if(!now->ls) now->ls = new Seg(l,m);
            self(self,now->ls,l,m);
        }
        if(qr > m){
            if(!now->rs) now->rs = new Seg(m,r);
            self(self,now->rs,m,r);
        }
    };
    find(find,st,-ninf,ninf);
};
//Seg *st = new Seg(-ninf,ninf); // [l,r)
```

3.6 SegTree Beats/線段樹 Beats

```
//阿傑不知道這是啥
struct SegTreeBeat{
    struct node{int mx,smx,cntmx,sum;};
    #define cl(x) (x<<1)+1
    #define cr(x) (x<<1)+2
    #define mid ((l+r)>>1)
    vector<node> seg;
    vector<int> tag;
    vector<int> *a;
    int n;
    void pull(int id){
        seg[id].sum=seg[cl(id)].sum+seg[cr(id)].sum;

    if(seg[cl(id)].mx==seg[cr(id)].mx){
        seg[id].mx=seg[cl(id)].mx;
        seg[id].smx=max(seg[cl(id)].smx,seg[cr(id)].smx
            );
        seg[id].cntmx=seg[cl(id)].cntmx+seg[cr(id)].
            cntmx;
    }else if(seg[cl(id)].mx>seg[cr(id)].mx){
        seg[id].mx=seg[cl(id)].mx;
        seg[id].smx=max(seg[cl(id)].smx,seg[cr(id)].mx)
            ;
        seg[id].cntmx=seg[cl(id)].cntmx;
    }else{
        seg[id].mx=seg[cr(id)].mx;
        seg[id].smx=max(seg[cr(id)].smx,seg[cl(id)].mx)
            ;
        seg[id].cntmx=seg[cr(id)].cntmx;
    }}

    void init(vector<int> &v){
        n=v.size();
        seg.resize(n*4+5);
        tag=vector<int>(n*4+5,0);
        a=&v;
    }
    void build(int l,int r,int id=0){
        if(l==r){
            seg[id]={(*a)[l],LLONG_MIN,1,(*a)[l]};
            return;
        }
        build(l,mid,cl(id));
        build(mid+1,r,cr(id));
        pull(id);
    }
    void apply_chmin(int id,int v){
        if(seg[id].mx<=v) return;
        seg[id].sum-=(seg[id].mx-v)*seg[id].cntmx;
        seg[id].mx=v;
    }
    void apply_add(int id,int l,int r,int v){
        seg[id].sum+=v*(r-l+1);
        if(seg[id].smx!=LLONG_MIN) seg[id].smx+=v;
        seg[id].mx+=v;
        tag[id]+=v;
    }
    void push(int id,int l,int r){
        if(l==r) return;
        if(tag[id]){
            apply_add(cl(id),l,mid,tag[id]);
            apply_add(cr(id),mid+1,r,tag[id]);
            tag[id]=0;
        }
        apply_chmin(cl(id),seg[id].mx);
        apply_chmin(cr(id),seg[id].mx);
    }
    void add(int l,int r,int sl,int sr,int v,int id=0){
        if(sl<=l && r<=sr){
            apply_add(id,l,r,v);
            return;
        }
        push(id,l,r);
        if(sl<=mid) add(l,mid,sl,sr,v,cl(id));
        if(mid<sr) add(mid+1,r,sl,sr,v,cr(id));
        pull(id);
    }
    void chmin(int l,int r,int sl,int sr,int v,int id
        =0){
        if(seg[id].mx<=v) return;
        if(sl<=l && r<=sr && seg[id].smx<v){
            apply_chmin(id,v);
            return;
        }
        push(id,l,r);
        if(sl<=mid) chmin(l,mid,sl,sr,v,cl(id));
        if(mid<sr) chmin(mid+1,r,sl,sr,v,cr(id));
        pull(id);
    }
    int qry(int l,int r,int sl,int sr,int id=0){
        if(sl<=l && r<=sr) return seg[id].sum;
        push(id,l,r);
        int ans=0;
        if(sl<=mid) ans+=qry(l,mid,sl,sr,cl(id));
        if(mid<sr) ans+=qry(mid+1,r,sl,sr,cr(id));
        return ans;
}};
```

4 Graph

4.1 Topo/拓樸

```
vector<int> edge[MAXN], ans;
int deg[MAXN];

void topo(int n) {
    queue<int> q;

    for (int i = 1; i <= n; i++)
        if (!deg[i])
```

```
            q.push(i);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);

        for (int v : edge[u]) {
            deg[v]--;
            if (!deg[v])
                q.push(v);
        }
    }
}
```

4.2 bellman/邊長鬆弛

```
struct Edge {//1-base
  int from, to, weight;
} edge[MAXN];
int dis[MAXN];
void bellmen_ford(int n, int m) {
  dis[1] = 0;  // 起點的距離一定是 0
  for (int j = 0; j < n - 1;
       j++) {  // n 個節點要跑 n - 1 次
    for (int i = 0; i < m; i++) {  // 對於所有邊都嘗試
        鬆弛
      if (dis[edge[i].to] >
          dis[edge[i].from] + edge[i].weight)
        dis[edge[i].to] =
            dis[edge[i].from] + edge[i].weight;
    }
  }
}
```

4.3 ola/歐拉

```
//歐拉路徑(無向圖)兩個奇數點或者全偶點
//歐拉路徑(有向圖)條件:起點out=in+1,終點in=out+1,其他點
    in=out
//歐拉迴路(無向圖)所有點偶數點
//歐拉迴路(有向圖)所有點in=out
vector<int> anse;//邊
vector<int> ansp;//點
vector<bool> vis(m,false);
function<void(int)> dfs=[&](int u){
    while(!mp[u].empty()){
        auto [x,e]=mp[u].back();
        mp[u].pop_back();
        if(vis[e]){
            continue;
        }
        vis[e]=true;
        dfs(x);
        anse.pb(e);
    }
    ansp.pb(u);
};
```

4.4 LCA/最近公共祖先

```
struct{
    int n, logN;
    vector<vector<int>>jump;
    vector<int>in, out;
    bool is_ancestor(int x, int y){
        if(x == -1 || y == -1) return 1;
        return in[x] <= in[y] && out[y] <= out[x];
    }
    int query(int x, int y){
        if(is_ancestor(x, y)) return x;
        if(is_ancestor(y, x)) return y;
        for(int i = logN - 1; i >= 0; i--){
            if(!is_ancestor(jump[x][i], y)) x = jump[x
                ][i];
        }
        return jump[x][0];
    }
    void init(int _n, vector<vector<int>>&v){//size,
        adj array 0base
        n = _n;
        logN = log2(n) + 1;

        jump.resize(n, vector<int>(logN, -1));
        in.resize(n); out.resize(n);

        int timing = 1;
        auto dfs = [&](auto &&self, int x) -> void{
            in[x] = timing++;
            for(auto &i : v[x]){
                jump[i][0] = x;
                self(self, i);
            }
            out[x] = timing++;
            return;
        }; dfs(dfs, 0);

        for(int i = 1; i < logN; i++){
            for(int now = 1; now < n; now++){
                if(jump[now][i-1] == -1) continue;
                jump[now][i] = jump[jump[now][i-1]][i
                    -1];
            }
        }
    }
}LCA;
```

4.5 Scc/強連通分量

```
//同一團u,v之間有路徑
//bln:在哪個分輛 E原圖
struct Scc{
  int n, nScc, vst[MXN], bln[MXN];
  vector<int> E[MXN], rE[MXN], vec;
  void init(int _n){
    n = _n;
    for (int i=0; i<= n; i++)
      E[i].clear(), rE[i].clear();
  }
  void addEdge(int u, int v){
    E[u].PB(v); rE[v].PB(u);
  }
  void DFS(int u){
    vst[u]=1;
    for (auto v : E[u]) if (!vst[v]) DFS(v);
    vec.PB(u);
  }
  void rDFS(int u){
    vst[u] = 1; bln[u] = nScc;
    for (auto v : rE[u]) if (!vst[v]) rDFS(v);
  }
  void solve(){
    nScc = 0;
    vec.clear();
    fill(vst, vst+n+1, 0);
    for (int i=0; i<n; i++)
      if (!vst[i]) DFS(i);
    reverse(vec.begin(),vec.end());
    fill(vst, vst+n+1, 0);
    for (auto v : vec)
      if (!vst[v]){
        rDFS(v); nScc++;
      }
  }
};
```

4.6 Bcc/雙連通分量

```
//雙連通分量定義:每個團塊刪除任意一個點都連通,每個割點
    都屬於多個雙連通分量
//割點出現數量>=2,割邊出現數量=1
struct BccVertex {
    int n, nScc, step, dfn[MXN], low[MXN];
    vector<int> E[MXN], sccv[MXN];
    int top, stk[MXN];
    void init(int _n) {
        n = _n;
        nScc = step = 0;
        for (int i = 0; i < n; i++) E[i].clear();
    }
    void addEdge(int u, int v) {
        E[u].PB(v);
        E[v].PB(u);
```

```
    }
    void DFS(int u, int f) {
        dfn[u] = low[u] = step++;
        stk[top++] = u;
        for (auto v : E[u]) {
            if (v == f) continue;
            if (dfn[v] == -1) {
                DFS(v, u);
                low[u] = min(low[u], low[v]);
                if (low[v] >= dfn[u]) {
                    int z;
                    sccv[nScc].clear();
                    do {
                        z = stk[--top];
                        sccv[nScc].PB(z);
                    } while (z != v);
                    sccv[nScc++].PB(u);
                }
            } else
                low[u] = min(low[u], dfn[v]);
        }
    }
    vector<vector<int>> solve() {
        vector<vector<int>> res;
        for (int i = 0; i < n; i++) dfn[i] = low[i] =
            -1;
        for (int i = 0; i < n; i++)
            if (dfn[i] == -1) {
                top = 0;
                DFS(i, i);
            }
        REP(i, nScc) res.PB(sccv[i]);
        return res;
    }
} graph;
```

4.7 最大團

```
//最大團定義:團內每個點都有一條邊連接到其他點
//O(1.4422^n)
#define N 111
struct MaxClique{ // 0-base
  typedef bitset<N> Int;
  Int linkto[N] , v[N];
  int n;
  void init(int _n){
    n = _n;
    for(int i = 0 ; i < n ; i ++){
      linkto[i].reset(); v[i].reset();
  } }
  void addEdge(int a , int b)
  { v[a][b] = v[b][a] = 1; }
  int popcount(const Int& val)
  { return val.count(); }
  int lowbit(const Int& val)
  { return val._Find_first(); }
  int ans , stk[N];
  int id[N] , di[N] , deg[N];
  Int cans;
  void maxclique(int elem_num, Int candi){
    if(elem_num > ans){
      ans = elem_num; cans.reset();
      for(int i = 0 ; i < elem_num ; i ++)
        cans[id[stk[i]]] = 1;
    }
    int potential = elem_num + popcount(candi);
    if(potential <= ans) return;
    int pivot = lowbit(candi);
    Int smaller_candi = candi & (~linkto[pivot]);
    while(smaller_candi.count() && potential > ans){
      int next = lowbit(smaller_candi);
      candi[next] = !candi[next];
      smaller_candi[next] = !smaller_candi[next];
      potential --;
      if(next == pivot || (smaller_candi & linkto[next
          ]).count()){
        stk[elem_num] = next;
        maxclique(elem_num + 1, candi & linkto[next]);
  } } }
  int solve(){
    for(int i = 0 ; i < n ; i ++){
      id[i] = i; deg[i] = v[i].count();
    }
    sort(id , id + n , [&](int id1, int id2){
          return deg[id1] > deg[id2]; });
    for(int i = 0 ; i < n ; i ++) di[id[i]] = i;
    for(int i = 0 ; i < n ; i ++)
      for(int j = 0 ; j < n ; j ++)
        if(v[i][j]) linkto[di[i]][di[j]] = 1;
    Int cand; cand.reset();
    for(int i = 0 ; i < n ; i ++) cand[i] = 1;
    ans = 1;
    cans.reset(); cans[0] = 1;
    maxclique(0, cand);
    return ans;
} }solver;
/*獨立集 與 團 的概念完全相反 最大獨立集去建反圖之後，
    求最大團即可得到答案
獨立集定義: 獨立集內的任意兩個點之間都沒有邊相連
*/
```

4.8 平均最小團

```
/* minimum mean cycle O(VE) */
//求最小平均權重圈，若無圈則回傳inf
//定義: 圈的平均權重 = 圈的總權重 / 圈的邊數
struct MMC{
#define E 101010
#define V 1021
#define inf 1e9
#define eps 1e-6
  struct Edge { int v,u; double c; };
  int n, m, prv[V][V], prve[V][V], vst[V];
  Edge e[E];
  vector<int> edgeID, cycle, rho;
  double d[V][V];
  void init( int _n )
  { n = _n; m = 0; }
  // WARNING: TYPE matters
  void addEdge( int vi , int ui , double ci )
  { e[ m ++ ] = { vi , ui , ci }; }
  void bellman_ford() {
    for(int i=0; i<n; i++) d[0][i]=0;
    for(int i=0; i<n; i++) {
      fill(d[i+1], d[i+1]+n, inf);
      for(int j=0; j<m; j++) {
        int v = e[j].v, u = e[j].u;
        if(d[i][v]<inf && d[i+1][u]>d[i][v]+e[j].c) {
          d[i+1][u] = d[i][v]+e[j].c;
          prv[i+1][u] = v;
          prve[i+1][u] = j;
  } } } }
  double solve(){
    // returns inf if no cycle, mmc otherwise
    double mmc=inf;
    int st = -1;
    bellman_ford();
    for(int i=0; i<n; i++) {
      double avg=-inf;
      for(int k=0; k<n; k++) {
        if(d[n][i]<inf-eps) avg=max(avg,(d[n][i]-d[k][i
            ])/(n-k));
        else avg=max(avg,inf);
      }
      if (avg < mmc) tie(mmc, st) = tie(avg, i);
    }
    fill(vst, vst + sizeof(vst) / sizeof(vst[0]),0LL);
        edgeID.clear(); cycle.clear(); rho.clear();
    for (int i=n; !vst[st]; st=prv[i--][st]) {
      vst[st]++;
      edgeID.pb(prve[i][st]);
      rho.pb(st);
    }
    while (vst[st] != 2) {
      if(rho.empty()) return inf;
      int v = rho.back(); rho.pop_back();
      cycle.pb(v);
      vst[v]++;
    }
    reverse(edgeID.begin(),edgeID.end());
    edgeID.resize(cycle.size());
    return mmc;
} }mmc;
```

4.9 支配點

```
//支配點
//定義: s支配u <=> s到u的所有路徑都經過s
//最後 idom[u] 為點 u 的支配點
struct DominatorTree{ // O(N)
#define REP(i,s,e) for(int i=(s);i<=(e);i++)
#define REPD(i,s,e) for(int i=(s);i>=(e);i--)
  int n , s;
  vector< int > g[ MAXN ] , pred[ MAXN ];
  vector< int > cov[ MAXN ];
  int dfn[ MAXN ] , nfd[ MAXN ] , ts;
  int par[ MAXN ]; //idom[u] s到u的最後一個必經點
  int sdom[ MAXN ] , idom[ MAXN ];
  int mom[ MAXN ] , mn[ MAXN ];
  inline bool cmp( int u , int v )
  { return dfn[ u ] < dfn[ v ]; }
  int eval( int u ){
    if( mom[ u ] == u ) return u;
    int res = eval( mom[ u ] );
    if(cmp( sdom[ mn[ mom[ u ] ] ] , sdom[ mn[ u ] ] ))
      mn[ u ] = mn[ mom[ u ] ];
    return mom[ u ] = res;
  }
  void init( int _n , int _s ){
    ts = 0; n = _n; s = _s;
    REP( i, 1, n ) g[ i ].clear(), pred[ i ].clear(),
        dfn[i]=0;;

  }
  void addEdge( int u , int v ){
    g[ u ].push_back( v );
    pred[ v ].push_back( u );
  }
  void dfs( int u ){
    ts++;
    dfn[ u ] = ts;
    nfd[ ts ] = u;
    for( int v : g[ u ] ) if( dfn[ v ] == 0 ){
      par[ v ] = u;
      dfs( v );
  } }
  void build(){
    REP( i , 1 , n ){
      dfn[ i ] = nfd[ i ] = 0;
      cov[ i ].clear();
      mom[ i ] = mn[ i ] = sdom[ i ] = i;
    }
    dfs( s );
    REPD( i , n , 2 ){
      int u = nfd[ i ];
      if( u == 0 ) continue ;
      for( int v : pred[ u ] ) if( dfn[ v ] ){
        eval( v );
        if( cmp( sdom[ mn[ v ] ] , sdom[ u ] ) )
          sdom[ u ] = sdom[ mn[ v ] ];
      }
      cov[ sdom[ u ] ].push_back( u );
      mom[ u ] = par[ u ];
      for( int w : cov[ par[ u ] ] ){
        eval( w );
        if( cmp( sdom[ mn[ w ] ] , par[ u ] ) )
          idom[ w ] = mn[ w ];
        else idom[ w ] = par[ u ];
      }
      cov[ par[ u ] ].clear();
    }
    REP( i , 2 , n ){
      int u = nfd[ i ];
      if( u == 0 ) continue ;
      if( idom[ u ] != sdom[ u ] )
        idom[ u ] = idom[ idom[ u ] ];
}idom[s]=s; } }domT;
```

5 dynamic programming

5.1 SOS DP

```
// 求子集和 或超集和 -> !(mask & (1 << i))
for(int i = 0; i<(1<<N); ++i) F[i] = A[i]; //預處理 狀
    態權重

for(int i = 0;i < N; ++i)
for (int s = 0; s < (1<<N) ; ++s)
  if (s & (1 << i))
    F[s] += F[s ^ (1 << i)];

//窮舉子集合
for(int s = mask; s ; s = (s-1)&mask;)
//放這個做啥我不會用QAQ
```

5.2 p-median

```
void p_Median(){
    for (int i=1; i<=N; ++i)
        for (int j=i; j<=N; ++j){
            m = (i+j)/2,d[i][j] = 0;        // m是中位
                數，d[i][j]為距離的總和
            for (int k=i; k<=j; ++k) d[i][j] += abs(arr
                [k] - arr[m]);
        }
    for (int p=1; p<=P; ++p)
        for (int n=1; n<=N; ++n){
            dp[p][n] = 1e9;
            for (int k=p; k<=n; ++k)
                if (dp[p-1][k-1] + d[k][n] < dp[p][n]){
                    dp[p][n] = dp[p-1][k-1] + d[k][n];
                    r[p][n] = k;    // 從第k個位置往右
                        到第j個位置
                }
        }
}
```

6 Computational Geometry

6.1 defult

```
int sgn(double x) { return (x > -eps) - (x < eps); }
template<class T>
struct pt{
  T x,y;
  pt(T _x,T _y):x(_x),y(_y){}
  pt():x(0),y(0){}
  pt operator * (T  c){ return pt(x * c, y * c); }
  pt operator / (T  c){ return pt(x / c, y / c); }
  pt operator + (pt a){ return pt(x + a.x, y + a.y); }
  pt operator - (pt a){ return pt(x - a.x, y - a.y); }
  T  operator * (pt a){ return x * a.x + y * a.y; } //
      內積
  T  operator ^ (pt a){ return x * a.y - y * a.x; } //
      外積
    bool operator < (pt a) const { return x < a.x || (x
        == a.x && y < a.y); }//按照x排
    bool operator==(const pt<T>& a) const { return x ==
        a.x && y == a.y; }
};
template<class T>
ld abs(pt<T> a) {//返回向量長度
    return sqrt((ld)(a.x * a.x + a.y * a.y));
}
template<class T>
struct Line {//直線
    pt<T> a, b;
    pt<T> dir()const{return b-a;}
};
template<class T>//線段
T ori(pt<T> a, pt<T> b, pt<T> c){ return (b - a) ^ (c -
    a); }
template<class T>//判斷點是否在線段上
bool PtOnSeg(pt<T> p, Line<T> L) {
    return sgn(ori(L.a, L.b, p)) == 0 and sgn((p - L.a)
        * (p - L.b)) <= 0;
}
```

6.2 Circle

```
struct Cir {
    Pt o;
    ld r;
};
bool disjunct(const Cir &a, const Cir &b) { //  分開 ||
    外切 -> 1
```

```
    return sgn(abs(a.o - b.o) - a.r - b.r) >= 0;
}
bool contain(const Cir &a, const Cir &b) {
    return sgn(a.r - b.r - abs(a.o - b.o)) >= 0;
}
```

6.3 極座標排序

```
//由+x軸往逆時針排序
struct point {
    int x, y;
};
long long cross(const point &a, const point &b) {
    return (long long) a.x * b.y - (long long) a.y * b.
        x;
}
bool cmp(const point &a, const point &b) {
    int ah = (a.y < 0 or (a.y == 0 and a.x < 0));
    int bh = (b.y < 0 or (b.y == 0 and b.x < 0));
    if (ah != bh) return ah < bh;
    return cross(a, b) > 0;
}
void argument_sort(vector<point> &points) {
    sort(points.begin(), points.end(), cmp);
}
```

6.4 ConvexHull/凸包 (傑)

```
template<class T>
vector<pt<T>> Hull(vector<pt<T>> P) {
    sort(all(P));
    P.erase(unique(all(P)), P.end());
    P.insert(P.end(), P.rbegin() + 1, P.rend());
    vector<pt<T>> stk;
    for (auto p : P) {
        auto it = stk.rbegin();
        while (stk.rend() - it >= 2 and \
            ori(*next(it), *it, p) <= 0 and \
            (*next(it) < *it) == (*it < p)) {
            it++;
        }
        stk.resize(stk.rend() - it);
        stk.push_back(p);
    }
    stk.pop_back();
    return stk;
}
```

6.5 ConvexHull/凸包 (宏)

```
struct ConvexHull{
    vector<Pt> vec;
    int n;
    void init(vector<Pt> &v){
        vec=v;
        n=v.size();
    }
    int cross(Pt o,Pt a,Pt b){return (a-o)^(b-o);}
    vector<int> point;
    vector<int> area;
    vector<Pt> up,down;
    void build(){
        up.clear();down.clear();
        point=vector<int>(n+1);
        area=vector<int>(n+1);
        int upsum=0;
        int downsum=0;
        for(int i=0;i<n;i++){
            while(up.size()>=2 &&
                  cross(up[up.size()-1],
                        up[up.size()-2],vec[i])<=0){
                upsum-=cross({0,0},up[up.size()-2],
                             up[up.size()-1]);
                up.pop_back();
            }
            up.push_back(vec[i]);
            if(up.size()>1)
                upsum+=cross({0,0},up[up.size()-2],
                             up[up.size()-1]);

            while(down.size()>=2 &&
                  cross(down[down.size()-1],
                        down[down.size()-2],vec[i])>=0)
                        {
                downsum-=cross({0,0},down[down.size()
                    -2],
                               down[down.size()-1]);
                down.pop_back();
            }
            down.push_back(vec[i]);
            if(down.size()>1)
                downsum+=cross({0,0},down[down.size()
                    -2],
                               down[down.size()-1]);

            area[i+1]=abs(upsum-downsum);

            if(i+1==1) point[i+1]=1;
            else point[i+1]=up.size()+down.size()-2;
    }}
    double getarea(int p){
        return area[p]/2.0;
    }
    int getnum(int p){
        return point[p];
    }
    vector<Pt> getall(){
        vector<Pt> ans;
        for(int i=0;i<down.size();i++)
            ans.push_back(down[i]);
        for(int i=up.size()-2;i>=1;i--)
            ans.push_back(up[i]);
        return ans;
    }
};
```

6.6 PtSegDist

```
template<class T>//計算點到線段的距離
ld PtSegDist(pt<T> p, Line<T> l) {
    ld ans = min(abs(p - l.a), abs(p - l.b));
    if (sgn(abs(l.a - l.b)) == 0)  return ans;
    if (sgn((l.a - l.b) * (p - l.b)) < 0) return ans;
        // case1
    if (sgn((l.b - l.a) * (p - l.a)) < 0) return ans;
        // case1
    return min(ans, abs(ori(p, l.a, l.b)) / abs(l.a - l
        .b)); // case2
}
```

6.7 兩線相交

```
bool isInter(Line l, Line m) {
    if (PtOnSeg(m.a, l) or PtOnSeg(m.b, l) or
        PtOnSeg(l.a, m) or PtOnSeg(l.b, m))
        return true;
    return PtSide(m.a, l) * PtSide(m.b, l) < 0 and
           PtSide(l.a, m) * PtSide(l.b, m) < 0;
}
Pt LineInter(Line l, Line m) {//交點
    double s = ori(m.a, m.b, l.a), t = ori(m.a, m.b, l.
        b);
    return (l.b * s - l.a * t) / (s - t);
}
```

6.8 點是否在多邊形內

```
template<class T>//判斷點是否在多邊形內
int inPoly(pt<T> p, const vector<pt<T>> &P) {
    const int n = P.size();
    int cnt = 0;
    for (int i = 0; i < n; i++) {
        pt<T> a = P[i], b = P[(i + 1) % n];
        if (PtOnSeg(p, {a, b})) return 1; // on edge
        if ((sgn(a.y - p.y) == 1) ^ (sgn(b.y - p.y) ==
            1))
            cnt += sgn(ori(a, b, p));
    }
    return cnt == 0 ? 0 : 2; // out, in
}
```

6.9 最遠點對/旋轉卡尺

```
double RotatingCalipers(vector<Pt> &v){ //逆時針
    int n=v.size();
    double dist=0;
    if(n==1) return 0;
    if(n==2) return dis(v[0]-v[1]);
    for(int i=0,j=2;i<n;i++){
        Pt a=v[i],b=v[(i+1)%n];
        while(cross(v[j],a,b)<cross(v[(j+1)%n],a,b))
            j=(j+1)%n;
        dist=max(dist,dis(v[j]-a));
        dist=max(dist,dis(v[j]-b));
    }
    return dist;
}
```

6.10 最近點對

```
//最近點對
//O(nlogn)
pair<ld, pair<i32, i32>> ClosestPair(vector<Pt> &P) {
    // ans = dis * dis  !!注意ans overflow問題
    if (P.size() == 1) { return {1e200L, {0, 0}}; }
    pair<i32, i32> ansi;
    auto ans = abs2(P[0] - P[1]);
    ansi = {0, 1};
    auto upd = [&](const Pt &a, const Pt &b) {
        auto dis = abs2(a - b);
        if (dis < ans) ans = dis, ansi.FF = a.id, ansi.
            SS = b.id;
    };
    auto cmpy = [](const Pt &a, const Pt &b) { return a
        .y < b.y; };

    vector<Pt> t(P.size() + 1);
    auto rec = [&](auto &&self, i32 l, i32 r) {
        if (r - l <= 3) {
            for (i32 i = l; i <= r; i++)
                for (i32 j = i + 1; j <= r; j++) upd(P[
                    i], P[j]);
            sort(P.begin() + l, P.begin() + r + 1, cmpy
                );
            return;
        }

        i32 m = (l + r) >> 1;
        auto midx = P[m].x;
        self(self, l, m), self(self, m + 1, r);
        i32 tsz = 0;
        inplace_merge(P.begin() + l, P.begin() + m + 1,
             P.begin() + r + 1, cmpy);
        for (i32 i = l; i <= r; i++) {
            if (abs(P[i].x - midx) * abs(P[i].x - midx)
                 >= ans) continue;
            for (i32 j = tsz - 1; j >= 0 && (P[i].y - t
                [j].y) * (P[i].y - t[j].y) < ans; j--)
                upd(P[i], t[j]);
            t[tsz++] = P[i];
        }
    };
    sort(all(P));
    rec(rec, 0, P.size() - 1);
    return make_pair(sqrt(ans), ansi);
}
```

6.11 最小覆蓋圓

```
/* minimum enclosing circle */
int n;
Pt p[ N ];
const Circle circumcircle(Pt a,Pt b,Pt c){
  Circle cir;
  double fa,fb,fc,fd,fe,ff,dx,dy,dd;
  if( iszero( ( b - a ) ^ ( c - a ) ) ){
    if( ( ( b - a ) * ( c - a ) ) <= 0 )
      return Circle((b+c)/2,norm(b-c)/2);
    if( ( ( c - b ) * ( a - b ) ) <= 0 )
      return Circle((c+a)/2,norm(c-a)/2);
    if( ( ( a - c ) * ( b - c ) ) <= 0 )
      return Circle((a+b)/2,norm(a-b)/2);
  }else{
    fa=2*(a.x-b.x);
    fb=2*(a.y-b.y);
    fc=norm2(a)-norm2(b);
    fd=2*(a.x-c.x);
    fe=2*(a.y-c.y);
    ff=norm2(a)-norm2(c);
    dx=fc*fe-ff*fb;
    dy=fa*ff-fd*fc;
    dd=fa*fe-fd*fb;
    cir.o=Pt(dx/dd,dy/dd);
    cir.r=norm(a-cir.o);
    return cir;
  }
}
inline Circle mec(int fixed,int num){
  int i;
  Circle cir;
  if(fixed==3) return circumcircle(p[0],p[1],p[2]);
  cir=circumcircle(p[0],p[0],p[1]);
  for(i=fixed;i<num;i++) {
    if(cir.inside(p[i])) continue;
    swap(p[i],p[fixed]);
    cir=mec(fixed+1,i+1);
  }
  return cir;
}
inline double min_radius() {
  if(n<=1) return 0.0;
  if(n==2) return norm(p[0]-p[1])/2;
  scramble();
  return mec(0,n).r;
}
```

7 Flow

7.1 km/二分圖最大權匹配

```
  struct KM{ // max weight, for min negate the weights
  int n, mx[MXN], my[MXN], pa[MXN];
  ll g[MXN][MXN], lx[MXN], ly[MXN], sy[MXN];
  bool vx[MXN], vy[MXN];
  void init(int _n) { // 1-based, N個節點
    n = _n;
    for(int i=1; i<=n; i++) fill(g[i], g[i]+n+1, 0);
  }
  void addEdge(int x, int y, ll w) {g[x][y] = w;} //左
      邊的集合節點x連邊右邊集合節點y權重為w
  void augment(int y) {
    for(int x, z; y; y = z)
      x=pa[y], z=mx[x], my[y]=x, mx[x]=y;
  }
  void bfs(int st) {
    for(int i=1; i<=n; ++i) sy[i]=INF, vx[i]=vy[i]=0;
    queue<int> q; q.push(st);
    for(;;) {
      while(q.size()) {
        int x=q.front(); q.pop(); vx[x]=1;
        for(int y=1; y<=n; ++y) if(!vy[y]){
          ll t = lx[x]+ly[y]-g[x][y];
          if(t==0){
            pa[y]=x;
            if(!my[y]){augment(y);return;}
            vy[y]=1, q.push(my[y]);
          }else if(sy[y]>t) pa[y]=x,sy[y]=t;
      } }
      ll cut = INF;
      for(int y=1; y<=n; ++y)
        if(!vy[y]&&cut>sy[y]) cut=sy[y];
      for(int j=1; j<=n; ++j){
        if(vx[j]) lx[j] -= cut;
        if(vy[j]) ly[j] += cut;
        else sy[j] -= cut;
      }
      for(int y=1; y<=n; ++y) if(!vy[y]&&sy[y]==0){
        if(!my[y]){augment(y);return;}
        vy[y]=1, q.push(my[y]);
  } } }
  ll solve(){ // 回傳值為完美匹配下的最大總權重
    fill(mx, mx+n+1, 0); fill(my, my+n+1, 0);
    fill(ly, ly+n+1, 0); fill(lx, lx+n+1, -INF);
    for(int x=1; x<=n; ++x) for(int y=1; y<=n; ++y) //
        1-base
      lx[x] = max(lx[x], g[x][y]);
```

```
    for(int x=1; x<=n; ++x) bfs(x);
    ll ans = 0;
    for(int y=1; y<=n; ++y) ans += g[my[y]][y];
    return ans;
} }graph;
```

7.2 Dinic

```
struct Dinic{
  struct Edge{ int v,f,re; };
  int n,s,t,level[MXN];
  vector<Edge> E[MXN];
  void init(int _n, int _s, int _t){
    n = _n; s = _s; t = _t;
    for (int i=0; i<n; i++) E[i].clear();
  }
  void add_edge(int u, int v, int f){
    E[u].PB({v,f,SZ(E[v])});
    E[v].PB({u,0,SZ(E[u])-1});
  }
  bool BFS(){
    for (int i=0; i<n; i++) level[i] = -1;
    queue<int> que;
    que.push(s);
    level[s] = 0;
    while (!que.empty()){
      int u = que.front(); que.pop();
      for (auto it : E[u]){
        if (it.f > 0 && level[it.v] == -1){
          level[it.v] = level[u]+1;
          que.push(it.v);
    } } }
    return level[t] != -1;
  }
  int DFS(int u, int nf){
    if (u == t) return nf;
    int res = 0;
    for (auto &it : E[u]){
      if (it.f > 0 && level[it.v] == level[u]+1){
        int tf = DFS(it.v, min(nf,it.f));
        res += tf; nf -= tf; it.f -= tf;
        E[it.v][it.re].f += tf;
        if (nf == 0) return res;
    } }
    if (!res) level[u] = -1;
    return res;
  }
  int flow(int res=0){
    while ( BFS() )
      res += DFS(s,2147483647);
    return res;
} }flow;
/*flow.init(n, source, sink); : 初始化點數、源點、匯點
flow.add_edge(u, v, w); : 加入一條點u連到點v容量為w的邊
flow.flow(); : 回傳值為最大流量
*/
```

7.3 hungarian/匈牙利演算法

```
//二分圖最大匹配
//左邊有L個點，右邊有R個點
//使用前請先呼叫add_edge()加入邊
//匈牙利演算法 O(VE)
//與km演算法不同，匈牙利演算法只會回傳最大匹配數量，無
    法回傳最大權匹配
struct hungarian {
    int L, R;
    vector<bool> vis;
    vector<int> match;
    vector<vector<int>> E;
    hungarian(int l, int r): //左邊有幾個，右邊有幾個
    L(l), R(r),
    vis(l+r+1),
    match(l+r+1,-1),
    E(l+r+1){}
    void add_edge(int l, int r){//左側第幾個(1-base) ,
        右側第幾個(1-base)
        r = L+r;
        E[l].push_back(r);
        E[r].push_back(l);
    }
    bool dfs(int u){
        for(int i : E[u]){
            if(vis[i]) continue; // 有連通且未拜訪
            vis[i] = true; // 紀錄是否走過
            if(match[i] == -1 || dfs(match[i])){
                match[i] = u; match[u] = i; // 紀錄匹配
                return true;
            }
        }
        return false;
    }
    int solve(){
        int ans = 0;
        for(int i = 1; i <= L; i++){
            fill(all(vis),0);
            if(dfs(i)) ans++;
        }
        return ans;
    }
};
```

7.4 SPFA 最小費流

```
struct zkwflow{
  static const int maxN=10000;
  struct Edge{ int v,f,re; ll w;};
  int n,s,t,ptr[maxN]; bool vis[maxN]; ll dis[maxN];
  vector<Edge> E[maxN];
  void init(int _n,int _s,int _t){
    n=_n,s=_s,t=_t;
    for(int i=0;i<n;i++) E[i].clear();
  }
  void addEdge(int u,int v,int f,ll w){
    E[u].push_back({v,f,(int)E[v].size(),w});
    E[v].push_back({u,0,(int)E[u].size()-1,-w});
  }
  bool SPFA(){
    fill_n(dis,n,LLONG_MAX); fill_n(vis,n,false);
    queue<int> q; q.push(s); dis[s]=0;
    while (!q.empty()){
      int u=q.front(); q.pop(); vis[u]=false;
      for(auto &it:E[u]){
        if(it.f>0&&dis[it.v]>dis[u]+it.w){
          dis[it.v]=dis[u]+it.w;
          if(!vis[it.v]){
            vis[it.v]=true; q.push(it.v);
    } } } }
    return dis[t]!=LLONG_MAX;
  }
  int DFS(int u,int nf){
    if(u==t) return nf;
    int res=0; vis[u]=true;
    for(int &i=ptr[u];i<(int)E[u].size();i++){
      auto &it=E[u][i];
      if(it.f>0&&dis[it.v]==dis[u]+it.w&&!vis[it.v]){
        int tf=DFS(it.v,min(nf,it.f));
        res+=tf,nf-=tf,it.f-=tf;
        E[it.v][it.re].f+=tf;
        if(nf==0){ vis[u]=false; break; }
      }
    }
    return res;
  }
  pair<int,ll> flow(){
    int flow=0; ll cost=0;
    while (SPFA()){
      fill_n(ptr,n,0);
      int f=DFS(s,INT_MAX);
      flow+=f; cost+=dis[t]*f;
    }
    return{ flow,cost };
  } // reset: do nothing
} flow;
```

7.5 差分約束系統

- 2-SAT :
  $(a_i \vee a_j) = true \ \forall (i,j)$
  對於任意限制 $(x \vee y)$
  建兩條有向邊 (要多編號 $\neg x$)
  $x \to \neg y$ and $y \to \neg x$
  跑 scc
  scc.bln[x] < scc.bln[$\neg x$] $\Leftrightarrow$ x is true
  scc.bln[$\neg x$] < scc.bln[x] $\Leftrightarrow$ x is false

$\exists x$ which scc.bln[x] == scc.bln[$\neg x$] $\Leftrightarrow$ 無解

- 差分約束:
  n 個變數及 m 個約束條件
  求滿足所有 $x_j - x_i \leq b_k$ $(i, j \in [1, n], k \in [1, m])$
  的一組 x_1 ... x_n
  可轉成 $x_j - x_i \leq b_k \rightarrow x_j \leq x_i + b_k$
  結論就是使得所有 x_j 變小以滿足上式
  建邊跑 SPFA/Bellman
  要多建起點 s 連到所有 i 且邊權 0, dis[s] = 0
  有負環則無解，否則起點到所有 i 的距離為一組解
  $x_j - x_i \leq k \Rightarrow$ addEdge $i \xrightarrow{k} j$
  $x_j - x_i \geq k \Rightarrow$ addEdge $j \xrightarrow{-k} i$
  $x_j = x_i \Rightarrow$ addEdge $i \xrightarrow{0} j$ and $j \xrightarrow{0} i$

8 String

8.1 KMP

```
int failure[MXN];
vector<int> KMP(vector<int>& t, vector<int>& p){
    vector<int> ret;
    if (p.size() > t.size()) return {};
    for (int i=1, j=failure[0]=-1; i<p.size(); ++i){
        while (j >= 0 && p[j+1] != p[i])
            j = failure[j];
        if (p[j+1] == p[i]) j++;
        failure[i] = j;
    }
    for (int i=0, j=-1; i<t.size(); ++i){
        while (j >= 0 && p[j+1] != t[i])
            j = failure[j];
        if (p[j+1] == t[i]) j++;
        if (j == p.size()-1){
            ret.push_back( i - p.size() + 1 );
            j = failure[j];
    }}
    return ret;
}
/*
vector<int> prefunc(const string& s){
  int n = s.size();
  vector<int> pi(n);
  for(int i=1,j=0;i<n;++i){
    j = pi[i-1];
    while(j && s[j] != s[i]) j = pi[j-1]; //往下找藍框
    if(s[j] == s[i]) ++j; //相等會多一格
    pi[i] = j;
  }
  return pi;
}*/
```

8.2 Z-value

```
int z[MXN];
void Z_value(const string& s) {
    //z[i] = lcp(s[0...],s[i...])
  int i, j, left, right, len = s.size();
  left=right=0; z[0]=len;
  for(i=1;i<len;i++) {
    j=max(min(z[i-left],right-i),0LL);
    for(;i+j<len&&s[i+j]==s[j];j++);
    z[i]=j;
    if(i+z[i]>right) {
      right=i+z[i];
      left=i;
}   }   }
```

8.3 馬拉車

```
//記得s.resize(2*n+1)
//z_value_pal(&s[0],n,a.data());
void z_value_pal(char *s,int len,int *z){
  len=(len<<1)+1;
  for(int i=len-1;i>=0;i--)
    s[i]=i&1?s[i>>1]:'@';
  z[0]=1;
  for(int i=1,l=0,r=0;i<len;i++){
    z[i]=i<r?min(z[l+l-i],r-i):1;
    while(i-z[i]>=0&&i+z[i]<len&&s[i-z[i]]==s[i+z[i]])
        ++z[i];
    if(i+z[i]>r) l=i,r=i+z[i];
} }
```

8.4 minRotation/最小字典序旋轉

```
//rotate(begin(s),begin(s)+minRotation(s),end(s))
int minRotation(string s) {
  int a = 0, N = s.size(); s += s;
  rep(b,0,N) rep(k,0,N) {
    if(a+k == b || s[a+k] < s[b+k])
      {b += max(0LL, k-1LL); break;}
    if(s[a+k] > s[b+k]) {a = b; break;}
  } return a;
}
```

8.5 SAIS

```
// 此為後綴陣列模板
// 用法:
// 把要處理的字串轉成整數陣列傳入suffix_array()，會得到
    SA與H維結果
const int N = 500010;  // 有可能要在這就寫原本字串的兩
    倍長
struct SA{
#define REP(i,n) for ( int i=0; i<int(n); i++ )
#define REP1(i,a,b) for ( int i=(a); i<=int(b); i++ )
  bool _t[N*2];
  int _s[N*2], _sa[N*2], _c[N*2], x[N], _p[N], _q[N*2],
      hei[N], r[N];
  int operator [] (int i){ return _sa[i]; }
  void build(int *s, int n, int m){
    memcpy(_s, s, sizeof(int) * n);
    sais(_s, _sa, _p, _q, _t, _c, n, m);
    mkhei(n);
  }
  void mkhei(int n){
    REP(i,n) r[_sa[i]] = i;
    hei[0] = 0;
    REP(i,n) if(r[i]) {
      int ans = i>0 ? max(hei[r[i-1]] - 1, 0) : 0;
      while(_s[i+ans] == _s[_sa[r[i]-1]+ans]) ans++;
      hei[r[i]] = ans;
    }
  }
  void sais(int *s, int *sa, int *p, int *q, bool *t,
      int *c, int n, int z){
    bool uniq = t[n-1] = true, neq;
    int nn = 0, nmxz = -1, *nsa = sa + n, *ns = s + n,
        lst = -1;
#define MS0(x,n) memset((x),0,n*sizeof(*(x)))
#define MAGIC(XD) MS0(sa, n); \
    memcpy(x, c, sizeof(int) * z); \
    XD; \
    memcpy(x + 1, c, sizeof(int) * (z - 1)); \
    REP(i,n) if(sa[i] && !t[sa[i]-1]) sa[x[s[sa[i
        ]-1]]++] = sa[i]-1; \
    memcpy(x, c, sizeof(int) * z); \
    for(int i = n - 1; i >= 0; i--) if(sa[i] && t[sa[i
        ]-1]) sa[--x[s[sa[i]-1]]] = sa[i]-1;
    MS0(c, z);
    REP(i,n) uniq &= ++c[s[i]] < 2;
    REP(i,z-1) c[i+1] += c[i];
    if (uniq) { REP(i,n) sa[--c[s[i]]] = i; return; }
    for(int i = n - 2; i >= 0; i--) t[i] = (s[i]==s[i
        +1] ? t[i+1] : s[i]<s[i+1]);
    MAGIC(REP1(i,1,n-1) if(t[i] && !t[i-1]) sa[--x[s[i
        ]]]=p[q[i]=nn++]=i);
    REP(i, n) if (sa[i] && t[sa[i]] && !t[sa[i]-1]) {
      neq=lst<0||memcmp(s+sa[i],s+lst,(p[q[sa[i]]+1]-sa
          [i])*sizeof(int));
      ns[q[lst=sa[i]]]=nmxz+=neq;
    }
    sais(ns, nsa, p + nn, q + n, t + n, c + z, nn, nmxz
        + 1);
    MAGIC(for(int i = nn - 1; i >= 0; i--) sa[--x[s[p[
        nsa[i]]]]] = p[nsa[i]]);
  }
}sa;
int H[ N ], SA[ N ];  // SA: 後綴陣列的length
                      // H[i]: SA[i]與SA[i -1]的共同子字串
                          長(LCP)
```

```
void suffix_array(int* ip, int len) {
  // should padding a zero in the back
  // ip is int array, len is array length
  // ip[0..n-1] != 0, and ip[len] = 0
  ip[len++] = 0;
  sa.build(ip, len, 128);
  for (int i = 0; i < len; i++) {
    H[i] = sa.hei[i + 1];
    SA[i] = sa._sa[i + 1];
  }
  // resulting height, sa array \in [0,len)
}
```

8.6 Trie/字典樹

```
struct trie{
    trie *nxt[26];
    int cnt;    //紀錄有多少個字串以此節點結尾
    int sz;     //有多少字串的前綴包括此節點
    trie():cnt(0),sz(0){
        memset(nxt,0,sizeof(nxt));
    }
};

//創建新的字典樹
trie *root = new trie();
// O(|s|)
void insert(string& s){
    trie *now = root;  // 每次從根結點出發
    for(auto i:s){
        now->sz++;
        if(now->nxt[i-'a'] == NULL){
            now->nxt[i-'a'] = new trie();
        }
        now = now->nxt[i-'a'];  //走到下一個字母
    }
    now->cnt++;
    now->sz++;
}
    // O(|s|)
int query_prefix(string& s){  //查詢有多少前綴為 s
    trie *now = root;    // 每次從根結點出發
    for(auto i:s){
        if(now->nxt[i-'a'] == NULL){
            return 0;
        }
        now = now->nxt[i-'a'];
    }
    return now->sz;
}
int query_count(string& s){  //查詢字串 s 出現次數
    trie *now = root;    // 每次從根結點出發
    for(auto i:s){
        if(now->nxt[i-'a'] == NULL){
            return 0;
        }
        now = now->nxt[i-'a'];
    }
    return now->cnt;
}
```

8.7 ACautomata/AC 自動機

```
//makefail before query
//用於多模式字串匹配
struct ACautomata{
  struct Node{
    int cnt,i;
    Node *go[26], *fail, *dic;
    Node (){
      cnt = 0; fail = 0; dic = 0; i = 0;
      memset(go,0,sizeof(go));
    }
  }pool[1048576],*root;
  int ans[1048576];
  int nMem,n_pattern;
  Node* new_Node(){
    pool[nMem] = Node();
    return &pool[nMem++];
  }
  void init() {
    nMem=0;root=new_Node();n_pattern=0;
    add("");
  }
  void add(const string &str) { insert(root,str,0); }
  void insert(Node *cur, const string &str, int pos){
    for(int i=pos;i<str.size();i++){
      if(!cur->go[str[i]-'a'])
        cur->go[str[i]-'a'] = new_Node();
      cur=cur->go[str[i]-'a'];
    }
    cur->cnt++;
    cur->i=n_pattern++;
  }
  void make_fail(){
    queue<Node*> que;
    que.push(root);
    while (!que.empty()){
      Node* fr=que.front(); que.pop();
      for (int i=0; i<26; i++){
        if (fr->go[i]){
          Node *ptr = fr->fail;
          while (ptr && !ptr->go[i]) ptr = ptr->fail;
          fr->go[i]->fail=ptr=(ptr?ptr->go[i]:root);
          fr->go[i]->dic=(ptr->cnt?ptr:ptr->dic);
          que.push(fr->go[i]);
  } } } }
  void query(string s){
      Node *cur=root;
      for(int i=0;i<(int)s.size();i++){
          while(cur&&!cur->go[s[i]-'a']) cur=cur->fail;
          cur=(cur?cur->go[s[i]-'a']:root);
          if(cur->i>=0) ans[cur->i]++;
          for(Node *tmp=cur->dic;tmp;tmp=tmp->dic)
              ans[tmp->i]++;
  }}// ans[i] : number of occurrence of pattern i
}AC;
```

9 Math

9.1 basic

```
[a,b] 有包含
(a,b) 沒包含
ll mul(ll x,ll y,ll mod){
  ll ret=x*y-(ll)((long double)x/mod*y)*mod;
  // LL ret=x*y-(LL)((long double)x*y/mod+0.5)*mod;
  return ret<0?ret+mod:ret;
}
ll add(int x,int y,int mod){
    return (x+y)%mod;
}
```

9.2 快速冪

```
int ksm(int x, int y) {
    int ans = 1;
    while (y) {
        if (y & 1)
            ans = ans * x;
        x = x * x;
        y >>= 1;
    }
    return ans;
}

// 矩陣乘法
vector<vector<int>> mul(const vector<vector<int>> &A,
                        const vector<vector<int>> &B) {
    if (A.empty() || B.empty() || A[0].empty() || B[0].
        empty()) { return {};}
    int m = A.size();
    int k = A[0].size();
    int n = B[0].size();
    vector<vector<int>> C(m, vector<int>(n, 0));
    for (int i = 0; i < m; i++) {
    for (int p = 0; p < k; p++) if (A[i][p]) {
    for (int j = 0; j < n; j++) {
    C[i][j] = (C[i][j] + 1LL * A[i][p] * B[p][j])%MOD;
}}}
    return C;
}
```

```
// 矩陣快速冪
vector<vector<int>> ksm(vector<vector<int>> base, int
    exp) {
    int n = base.size();
    vector<vector<int>> res(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (exp > 0) {
        if (exp & 1) res = mul(res, base);
        base = mul(base, base);
        exp >>= 1;
    }
    return res;
}
```

9.3 大數運算

```
//加法
string add(string num1, string num2) {
    string res = "";
    int i = num1.length() - 1, j = num2.length() - 1,
        carry = 0;
    while (i >= 0 || j >= 0 || carry) {
        int sum = carry + (i >= 0 ? num1[i--] - '0' :
            0) + (j >= 0 ? num2[j--] - '0' : 0);
        carry = sum / 10;
        res += to_string(sum % 10);
    }
    reverse(res.begin(), res.end());
    return res;
}

//減法
string sub(string num1, string num2) {
    string res = "";
    int i = num1.length() - 1, j = num2.length() - 1,
        borrow = 0;
    while (i >= 0) {
        int sub = (num1[i--] - '0') - (j >= 0 ? num2[j
            --] - '0' : 0) - borrow;
        if (sub < 0) {
            sub += 10;
            borrow = 1;
        } else {
            borrow = 0;
        }
        res += to_string(sub);
    }
    while (res.length() > 1 && res.back() == '0') res.
        pop_back(); // 去除前導0
    reverse(res.begin(), res.end());
    return res;
}

//乘法
string mul(string num1, string num2) {
    int n = num1.size(), m = num2.size();
    vector<int> res(n + m, 0);

    for (int i = n - 1; i >= 0; i--) {
        for (int j = m - 1; j >= 0; j--) {
            int p = (num1[i] - '0') * (num2[j] - '0');
            int sum = p + res[i + j + 1];

            res[i + j + 1] = sum % 10;
            res[i + j] += sum / 10;
        }
    }
    string ans = "";
    for (int x : res) {
        if (!(ans.empty() && x == 0))  // 跳過前導0
            ans += (char)(x + '0');
    }
    return ans.empty() ? "0" : ans;
}
//除法
bool smaller(string a, string b) {
    if (a.size() != b.size()) return a.size() < b.size
        ();
    return a < b;
}
string div(string num1, string num2) {
    string cur = "", ans = "";
    for (char c : num1) {
        cur += c;
        // 去掉前導0
        while (cur.size() > 1 && cur[0] == '0') cur.
            erase(cur.begin());

        int x = 0;
        while (!smaller(cur, num2)) {
            cur = sub(cur, num2);
            x++;
        }
        ans += (char)(x + '0');
    }
    // 去掉前導0
    while (ans.size() > 1 && ans[0] == '0') ans.erase(
        ans.begin());
    return ans;
}
string mod(string num1, string num2) {//餘數
    string cur = "";

    for (char c : num1) {
        cur += c;
        while (cur.size() > 1 && cur[0] == '0') cur.
            erase(cur.begin());
        while (!smaller(cur, num2)) {
            cur = sub(cur, num2);
        }
    }
    return cur;
}
```

9.4 Prime

```
/* 12721, 13331, 14341, 75577, 123457, 222557, 556679
* 999983, 1097774749, 1076767633, 100102021, 999997771
* 1001010013, 1000512343, 987654361, 999991231
* 999888733, 98789101, 987777733, 999991921, 1010101333
* 1010102101, 1000000000039, 1000000000000037
* 2305843009213693951, 4611686018427387847
* 9223372036854775783, 18446744073709551557 */
int mu[ N ] , p_tbl[ N ];
vector<int> primes;
void sieve() {
  mu[ 1 ] = p_tbl[ 1 ] = 1;
  for( int i = 2 ; i < N ; i ++ ){
    if( !p_tbl[ i ] ){
      p_tbl[ i ] = i;
      primes.push_back( i );
      mu[ i ] = -1;
    }
    for( int p : primes ){
      int x = i * p;
      if( x >= M ) break;
      p_tbl[ x ] = p;
      mu[ x ] = -mu[ i ];
      if( i % p == 0 ){
        mu[ x ] = 0;
        break;
} } } }
vector<int> factor( int x ){
  vector<int> fac{ 1 };
  while( x > 1 ){
    int fn = SZ(fac), p = p_tbl[ x ], pos = 0;
    while( x % p == 0 ){
      x /= p;
      for( int i = 0 ; i < fn ; i ++ )
        fac.PB( fac[ pos ++ ] * p );
  } }
  return fac;
}
```

9.5 ExGCD

```
//ax+by=gcd(a,b)
PII gcd(int a, int b){
if(b == 0) return {1, 0};
PII q = gcd(b, a % b);
return {q.second, q.first - q.second * (a / b)};
```

```
}
```

9.6 Phi

```
//歐拉函數 phi(n) 是指<=n中與n互質的數的個數
int _phi(int n) {  // O(sqrtN)
  int res = n, a = n;
  for (int i = 2; i * i <= a; i++) {
    if (a % i == 0) {
      res = res / i * (i - 1);
      while (a % i == 0) a /= i;
    }
  }
  if (a > 1) res = res / a * (a - 1);
  return res;
}

int phi[MXN];  // 建表 最大 1e7
void phi_table(int n) {
  phi[1] = 1;
  for (int i = 2; i <= n; ++i) {
    if (phi[i]) continue;
    for (int j = i; j <= n; j += i) {
      if (phi[j] == 0) phi[j] = j;
      phi[j] = phi[j] / i * (i - 1);
    }
  }
}

int prime[350000]; ll phi[5000005];
bool v[5000005];
#define N 5000000
void quick_euler() {
    int cnt = 0;
    for(int i = 2; i <= N; ++i) {
        if(!v[i]) prime[++cnt] = i, phi[i] = i - 1; //
            若 i 是質數，(i) = i - 1
        for(int j = 1; i * prime[j] <= N && j <= cnt;
            ++j) {
            v[i * prime[j]] = 1; // 不是質數就篩掉
            if(i % prime[j] == 0){
                phi[i * prime[j]] = phi[i] * prime[j];
                break;
            }
            else phi[i * prime[j]] = phi[i] * (prime[j]
                - 1); // 積性函數性質用在這裡
        }
    }
}
```

9.7 Euler/歐拉函數

```
int prime[350000]; ULL phi[5000005];
bool v[5000005];
#define N 5000000
void quick_euler() {
    int cnt = 0;
    for(int i = 2; i <= N; ++i) {
        if(!v[i]) prime[++cnt] = i, phi[i] = i - 1; //
            若 i 是質數，(i) = i - 1
        for(int j = 1; i * prime[j] <= N && j <= cnt;
            ++j) {
            v[i * prime[j]] = 1; // 不是質數就篩掉
            if(i % prime[j] == 0){
                phi[i * prime[j]] = phi[i] * prime[j];
                break;
            }
            else phi[i * prime[j]] = phi[i] * (prime[j]
                - 1); // 積性函數性質用在這裡
        }
    }
}
```

9.8 miller rabin/判斷質數

```
// n < 4,759,123,141        3 :  2, 7, 61
// n < 1,122,004,669,633    4 :  2, 13, 23, 1662803
// n < 3,474,749,660,383          6 :  pirmes <= 13
// n < 2^64                       7 :
// 2, 325, 9375, 28178, 450775, 9780504, 1795265022
// Make sure testing integer is in range [2, n-2] if
// you want to use magic.
ll magic[]={};
bool witness(ll a,ll n,ll u,int t){
  if(!a) return 0;
  ll x=mypow(a,u,n);//ksm(x,y,mod)
  for(int i=0;i<t;i++) {
    ll nx=mul(x,x,n);
    if(nx==1&&x!=1&&x!=n-1) return 1;
    x=nx;
  }
  return x!=1;
}
bool miller_rabin(ll n) {
  int s=(7);
  // iterate s times of witness on n
  if(n<2) return 0;
  if(!(n&1)) return n == 2;
  ll u=n-1; int t=0;
  // n-1 = u*2^t
  while(!(u&1)) u>>=1, t++;
  while(s--){
    ll a=magic[s]%n;
    if(witness(a,n,u,t)) return 0;
  }
  return 1;
}
```

9.9 Pollard Rho/質因數分解

```
//回傳n的其中一個因數
// does not work when n is prime  O(n^(1/4))
#define LL __int128
LL f(LL x, LL c, LL mod){ return add(mul(x,x,mod),c,mod
    ); }
LL pollard_rho(LL n) {
    LL c = 1, x = 0, y = 0, p = 2, q, t = 0;
    while (t++ % 128 or gcd(p, n) == 1) {
        if (x == y) c++, y = f(x = 2, c, n);
        if (q = mul(p, abs(x-y), n)) p = q;
        x = f(x, c, n); y = f(f(y, c, n), c, n);
    }
    return gcd(p, n);
}
/*function<void(int)> fact=[&](int x){
        if(x==1) return;
        if(miller_rabin(x)){
            f.push_back(x);return;}
        int tmp=pollard_rho(x);
        fact(tmp);fact(x/tmp);};
*/
```

9.10 FFT/傅立葉轉換（多項式相乘）

```
const int MAXN = 262144<<1;
// (must be 2^k)
// before any usage, run pre_fft() first
//用法: pre_fft(); fft(n, a); mul(n, a.data(), m, b.
    data(), ans.data());
//答案在 ans[0..n+m-2]，n,m為a,b的長度
typedef long double ld;
typedef complex<ld> cplx; //real() ,imag()
const ld PI = acosl(-1);
const cplx I(0, 1);
cplx omega[MAXN+1];
void pre_fft(){
  for(int i=0; i<=MAXN; i++)
    omega[i] = exp(i * 2 * PI / MAXN * I);
}
// n must be 2^k
void fft(int n, cplx a[], bool inv=false){
  int basic = MAXN / n;
  int theta = basic;
  for (int m = n; m >= 2; m >>= 1) {
    int mh = m >> 1;
    for (int i = 0; i < mh; i++) {
      cplx w = omega[inv ? MAXN-(i*theta%MAXN)
                         : i*theta%MAXN];
      for (int j = i; j < n; j += m) {
        int k = j + mh;
        cplx x = a[j] - a[k];
        a[j] += a[k];
        a[k] = w * x;
```

```
    } }
    theta = (theta * 2) % MAXN;
  }
  int i = 0;
  for (int j = 1; j < n - 1; j++) {
    for (int k = n >> 1; k > (i ^= k); k >>= 1);
    if (j < i) swap(a[i], a[j]);
  }
  if(inv) for (i = 0; i < n; i++) a[i] /= n;
}
cplx arr[MAXN+1];
inline void mul(int _n,ll a[],int _m,ll b[],ll ans[]){
  int n=1,sum=_n+_m-1;
  while(n<sum)
    n<<=1;
  for(int i=0;i<n;i++) {
    double x=(i<_n?a[i]:0),y=(i<_m?b[i]:0);
    arr[i]=complex<double>(x+y,x-y);
  }
  fft(n,arr);
  for(int i=0;i<n;i++)
    arr[i]=arr[i]*arr[i];
  fft(n,arr,true);
  for(int i=0;i<sum;i++)
    ans[i]=(ll)(arr[i].real()/4+0.5);
}
```

9.11 josephus/約瑟夫問題

```
int josephus(int n, int m){ //n人每m次
    int ans = 0;
    for (int i=1; i<=n; ++i)
        ans = (ans + m) % i;
    return ans;
}
```

9.12 指數塔

```
ll mod(ll x, ll m) {
    return x >= m ? (x % m) + m : x;
}
ll solve(int idx, ll m) {
    if (m == 1) return 1;                    // 模數降為 1
    if (idx == n) return mod(a[idx], m); // 已經到達塔頂
    ll nxt = solve(idx + 1, phi[m]);
    return fpow(a[idx], nxt, m); // 快速冪計算當前層
}
```

9.13 BSGS

```
  // gcd(a, n) = 1, 算 a^x == b (mod n)
ll BSGS(ll a, ll b, ll n) {
    a %= n, b %= n;
    if (b == 1) return 0;     // a^0 = 1
    ll m = (ll)sqrt(n) + 1;  // 保險一點
    map<ll, ll> mp;

    // 拆成 a^{i * m} == b * a^j (mod n)
    ll baby = b;
    for (ll j = 0; j <= m; j++) {
        mp[baby] = j;
        baby = baby * a % n;
    }

    ll step = fpow(a, m, n);
    ll giant = step;
    for (ll i = 1; i <= m; i++) {  // 計算 (a^m)^i
        if (mp.count(giant)) {
            ll j = mp[giant], res = i * m - j;
            if (res >= 0) return res;
        }
        giant = giant * step % n;
    }

    return -1;
}
```

9.14 $x^2 \pmod p == a$

```
void calcH(LL &t, LL &h, const LL p) {
  LL tmp=p-1; for(t=0;(tmp&1)==0;tmp/=2) t++; h=tmp;
}
// solve equation x^2 mod p = a
bool solve(LL a, LL p, LL &x, LL &y) {
  if(p == 2) { x = y = 1; return true; }
  int p2 = p / 2, tmp = mypow(a, p2, p);
  if (tmp == p - 1) return false;
  if ((p + 1) % 4 == 0) {
    x=mypow(a,(p+1)/4,p); y=p-x; return true;
  } else {
    LL t, h, b, pb; calcH(t, h, p);
    if (t >= 2) {
      do {b = rand() % (p - 2) + 2;
      } while (mypow(b, p / 2, p) != p - 1);
      pb = mypow(b, h, p);
    } int s = mypow(a, h / 2, p);
    for (int step = 2; step <= t; step++) {
      int ss = (((LL)(s * s) % p) * a) % p;
      for(int i=0;i<t-step;i++) ss=mul(ss,ss,p);
      if (ss + 1 == p) s = (s * pb) % p;
      pb = ((LL)pb * pb) % p;
    } x = ((LL)s * a) % p; y = p - x;
  } return true;
}
```

9.15 Theorem

- 盧卡斯定理 (Lucas's Theorem) :
  對於 $n, m \in \mathbb{Z}^*$ 與質數 P，$C(m,n) \pmod P = \prod(C(m_i, n_i))$ 其中 m_i 為 m 在 P 進位下的第 i 位數字。

- 斯特林近似公式 (Stirling approximation) :
  $n! \approx \sqrt{2\pi n}(\frac{n}{e})^n e^{\frac{1}{12n}}$

- 第一類斯特林數 (Stirling Numbers of the first kind，大小為 n 的置換中有 k 個循環) :
  $S(n,k)$ = 多項式 $\prod_{i=0}^{n-1}(x+i)$ 中 x^k 的係數

- 第二類斯特林數 (Stirling Numbers of the second kind，將 n 個元素劃分為 k 個非空集合) :
  $S(n,k) = \frac{1}{k!}\sum_{j=0}^{k}(-1)^{k-j}\binom{k}{j}j^n$

- 皮克定理 (Pick's Theorem) : $A = i + b/2 - 1$
  A：多邊形面積、i：內部網格點數、b：邊上網格點數

- 卡塔蘭數 (Catalan number) : $C_n = \binom{2n}{n}/(n+1)$
  $C_n^{n+m} - C_{n+1}^{n+m} = (m+n)!\frac{n-m+1}{n+1}$ 對於 $n \geq m$
  $C_n = \frac{1}{n+1}\binom{2n}{n} = \frac{(2n)!}{(n+1)!n!}$
  $C_0 = 1$ 且 $C_{n+1} = 2(\frac{2n+1}{n+2})C_n$
  $C_0 = 1$ 且 $C_{n+1} = \sum_{i=0}^{n} C_i C_{n-i}$ 對於 $n \geq 0$

- 歐拉示性數 (Euler Characteristic):
  平面圖：$V - E + F - C = 1$
  凸多面體：$V - E + F = 2$
  V, E, F, C：分別為頂點數、邊數、面數（區域數）與連通分量數

- 基爾霍夫定理 / 矩陣樹定理 (Kirchhoff's theorem) :
  $A_{ii} = \deg(i), A_{ij} = (i,j) \in E \ ? -1 : 0$，刪除任意一行與一列後計算矩陣 A 的行列式 $\det(A)$

- 波利亞計數定理 (Polya's theorem) (c 為顏色數，m 為置換中循環大小的數量):
  $\frac{1}{m}\sum_{i=1}^{m} c^{\gcd(i,m)}$

- 伯恩賽德引理 (Burnside's Lemma):
  $|X/G| = \frac{1}{|G|}\sum_{g \in G}|X^g|$

- 錯排公式 (Derangements) (n 個人中，每個人都不在原來位置的組合數):
  $dp[0] = 1; dp[1] = 0;$
  $dp[i] = (i-1) * (dp[i-1] + dp[i-2]);$

- 貝爾數 (Bell Number，將 n 個人分組的方法總數) :
  $B_0 = 1$
  $B_n = \sum_{k=0}^{n} S(n,k)$ (第二類斯特林數)
  $B_{n+1} = \sum_{k=0}^{n}\binom{n}{k}B_k$

- 威爾遜定理 (Wilson's theorem) :
  $(p-1)! \equiv -1 \pmod p$

- 費馬小定理 (Fermat's little theorem) :
  $a^p \equiv a \pmod p$

- 歐拉降冪公式應用 (Euler's totient function):
  $A^{B^C} \pmod p = \text{pow}(A, \text{pow}(B, C, p-1)) \pmod p$

- 歐拉函數降冪公式 (Extended Euler's Theorem):
  $A^B \pmod C = A^{(B \bmod \phi(C)) + \phi(C)} \pmod C$

- 6 的倍數恆等式:
  $(a-1)^3+(a+1)^3+(-a)^3+(-a)^3=6a$
- 標準楊表 (Standard Young Tableaux, SYT):
  $\lambda=(\lambda_1\geq\cdots\geq\lambda_k)$，$\sum\lambda_i=n$ 記作 $\lambda\vdash n$
  $\lambda\vdash n$ 意指 λ 為 n 的整數分拆 (Integer Partition)，例如 $n=10,\lambda=(6,4)$ 此分拆可表示一種楊表形狀。
  楊表：第 1 列 λ_1 格，$\cdots$，第 k 列 λ_k 格的方格圖。
  標準楊表：每列從左到右遞增，每行從上到下遞增。
  若 T 為某一排列 (Permutation) 經 RSK 對應後的標準楊表，則此排列的最長下降子序列 (LDS) 與最長遞增子序列 (LIS) 長度分別對應到 T 的列數與行數。
- RSK 對應 (RSK Correspondence):
  一個排列與一對標準楊表 (P,Q) 具有雙射關係。
  P：排列跑 RSK 演算法的結果，通常為半標準楊表 (SSYT)。
  Q：可用來還原排列 (類似排列矩陣)。
- 鉤長公式 (Hook length formula，形狀為 λ 的標準楊表個數):
  $f^\lambda=\frac{n!}{\prod h_\lambda(i,j)}$
  $h_\lambda(i,j)=$ 滿足 $(x=i\vee y=j)\wedge(x,y)\geq(i,j)$ 且 (x,y) 落在形狀 λ 內的點對 (x,y) 數量。
  遞迴式：
  (i) $f^{(0,\cdots,0)}=1$
  (ii) $f^{(\lambda_1,\cdots,\lambda_m)}=\sum_{k=1}^m f^{(\lambda_1,\cdots,\lambda_k-1,\lambda_{k+1},\cdots,\lambda_m)}$
- 環狀相鄰不同色著色數 (Chromatic polynomial of a cycle graph):
  $f(n,k)=(k-1)^n+(-1)^n(k-1)$
  其中 n 為頂點數，k 為顏色數。

10 酷東西可能會用到

10.1 areaofrect/矩形聯集面積

```
struct AreaofRectangles{
#define cl(x) (x<<1)
#define cr(x) (x<<1|1)
    int n, id, sid;
    pair<int,int> tree[MXN<<3];   // count, area
    vector<int> ind;
    tuple<int,int,int,int> scan[MXN<<1];
    void puint(int i, int l, int r){
        if(tree[i].first)  tree[i].second = ind[r+1] -
            ind[l];//當前區間有被做加值
        else if(l != r){//沒有就直接拿兩個子節點的第二
            個數字
            int mid = (l+r)>>1;
            tree[i].second = tree[cl(i)].second + tree[
                cr(i)].second;
        }
        else    tree[i].second = 0;
    }
    void upd(int i, int l, int r, int ql, int qr, int v
        ){
        if(ql <= l && r <= qr){
            tree[i].first += v;
            puint(i, l, r); return;
        }
        int mid = (l+r) >> 1;
        if(ql <= mid) upd(cl(i), l, mid, ql, qr, v);
        if(qr > mid) upd(cr(i), mid+1, r, ql, qr, v);
        puint(i, l, r);
    }
    void init(int _n){
        n = _n; id = sid = 0;
        ind.clear(); ind.resize(n<<1);
        fill(tree, tree+(n<<2), make_pair(0, 0));
    }
    void addRectangle(int lx, int ly, int rx, int ry){
        ind[id++] = lx; ind[id++] = rx;
        scan[sid++] = make_tuple(ly, 1, lx, rx);
        scan[sid++] = make_tuple(ry, -1, lx, rx);
    }
    int solve(){
        sort(ind.begin(), ind.end());
        ind.resize(unique(ind.begin(), ind.end()) - ind
            .begin());//離散化
        sort(scan, scan + sid);
        int area = 0, pre = get<0>(scan[0]);
        for(int i = 0; i < sid; i++){
            auto [x, v, l, r] = scan[i];
            area += tree[1].second * (x-pre);//
            upd(1, 0, ind.size()-1, lower_bound(ind.
                begin(), ind.end(), l)-ind.begin(),
                lower_bound(ind.begin(),ind.end(),r)-
                ind.begin()-1, v);
            pre = x;
        }
        return area;
}   }rect;
```





